

Terraform 05

1. Watch the Terraform-05 video.

2. EXECUTE THE SCRIPT SHOWN IN THE VIDEO

—>

Terraform Remote state and state locking:

=====

We can store terraform configuration files and state file in github or any other repository but it is not good practise.

We use s3, terraform storage, hashicorp consul to store the state file.

State locking is used to lock the state file so that no two users can execute the state file at the same point of time.

Remote backend and state locking:

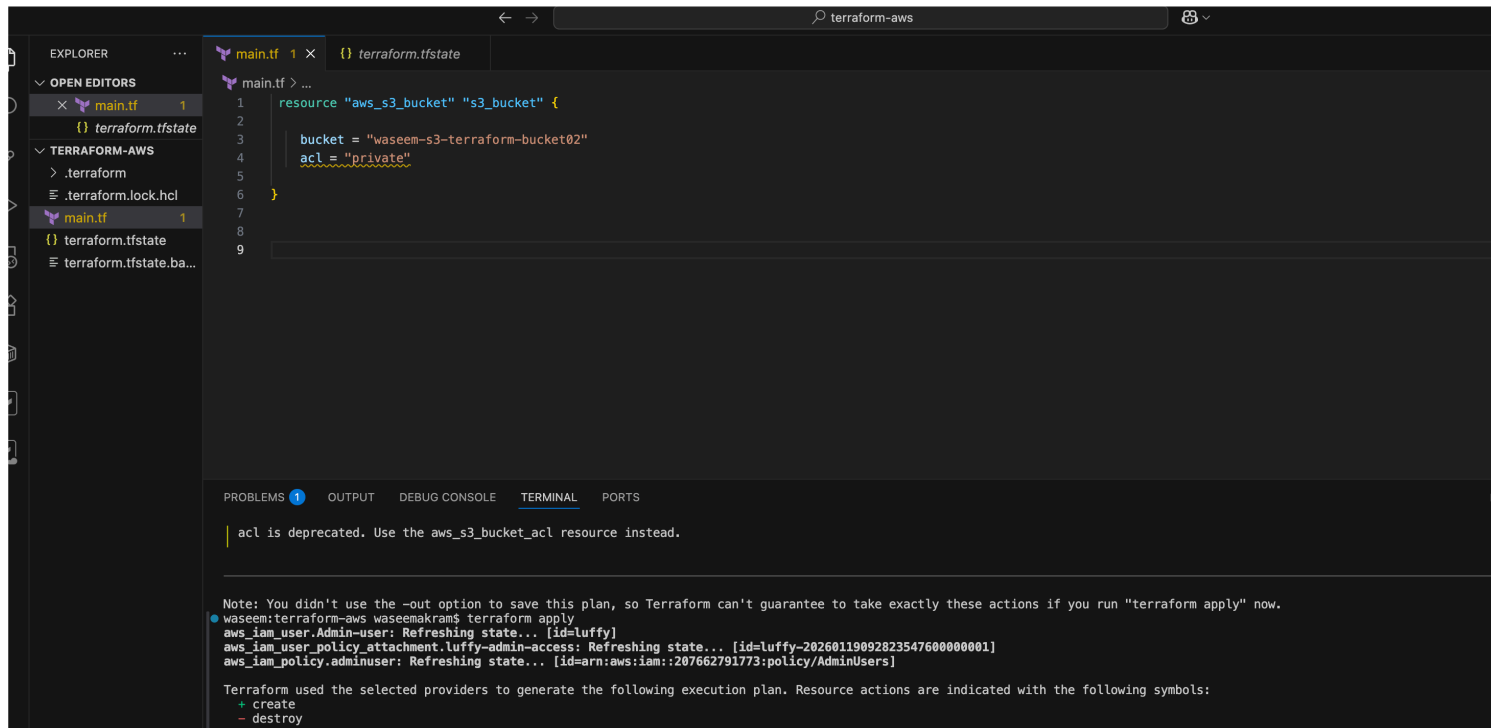
=====

We can use s3 as remote backend and dynamo db for state locking.

Create s3 using terraform:

=====

```
resource "aws_s3_bucket" "s3_bucket" {  
  
    bucket = "s3backend"  
    acl = "private"  
}
```



Plan: 1 to add, 0 to change, 2 to destroy.

Warning: Argument is deprecated

with `aws_s3_bucket.s3_bucket`,
on `main.tf` line 4, in resource `"aws_s3_bucket" "s3_bucket"`:
4: `acl = "private"`

`acl` is deprecated. Use the `aws_s3_bucket_acl` resource instead.

(and one more similar warning elsewhere)

Do you want to perform these actions?

Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

`aws_iam_policy.adminuser`: Destroying... [id=arn:aws:iam::207662791773:policy/AdminUsers]

`aws_iam_user.Admin-user`: Destroying... [id=luffy]

`aws_s3_bucket.s3_bucket`: Creating...

`aws_iam_user.Admin-user`: Destruction complete after 2s

`aws_iam_policy.adminuser`: Destruction complete after 2s

`aws_s3_bucket.s3_bucket`: Creation complete after 6s [id=waseem-s3-terraform-bucket02]

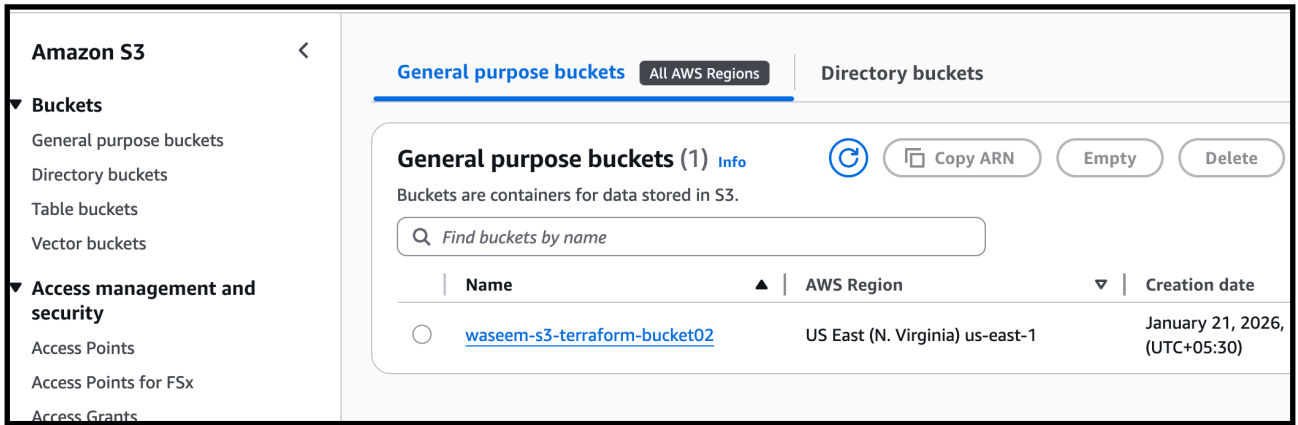
Warning: Argument is deprecated

with `aws_s3_bucket.s3_bucket`,
on `main.tf` line 4, in resource `"aws_s3_bucket" "s3_bucket"`:
4: `acl = "private"`

`acl` is deprecated. Use the `aws_s3_bucket_acl` resource instead.

Apply complete! Resources: 1 added, 0 changed, 2 destroyed.

○ waseem:terraform-aws waseemakram\$



Create dynamo db using terraform:

Create dynamo db using terraform:

```
resource "aws_dynamodb_table" "dynamodb-terraform-state-lock" {
  name = "terraform-state-lock-dynamo"
  hash_key = "LockID"
  read_capacity = 20
  write_capacity = 20

  attribute {
    name = "LockID"
    type = "S"
  }
}
```

```
main.tf 1 X
main.tf > ...
1 resource "aws_s3_bucket" "s3_bucket" {
2
3     bucket = "terraform-bucket-0880"
4     acl = "private"
5
6 }
7
8 resource "aws_dynamodb_table" "dynamodb-terraform-state-lock" {
9     name = "terraform-state-lock-dynamo"
10    hash_key = "LockID"
11    read_capacity = 20
12    write_capacity = 20
13
14    attribute {
15        name = "LockID"
16        type = "S"
17    }
18 }
19
20
21
```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS

(and one more similar warning elsewhere)

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

aws_s3_bucket.s3_bucket: Creating...
aws_s3_bucket.s3_bucket: Creation complete after 6s [id=terraform-bucket-0880]

Warning: Argument is deprecated

with aws_s3_bucket.s3_bucket,
on main.tf line 4, in resource "aws_s3_bucket" "s3_bucket":
4: acl = "private"

acl is deprecated. Use the aws_s3_bucket_acl resource instead.

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

- waseem:terraform-aws waseemakram\$ terraform init

Initializing the backend...
Initializing provider plugins...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v6.28.0

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see any changes that are required for your infrastructure. All Terraform commands should now work.

If you ever set or change modules or backend configuration for Terraform, rerun this command to reinitialize your working directory. If you forget, other commands will detect it and remind you to do so if necessary.

○ waseem:terraform-aws waseemakram\$

Ln 20, Col 1 Spaces: 4 UTF

—> first —> terraform init —>and then
— —> terraform apply

```
+ global_secondary_index (known after apply)
+ global_table_witness (known after apply)
+ point_in_time_recovery (known after apply)
+ server_side_encryption (known after apply)
+ ttl (known after apply)
+ warm_throughput (known after apply)
}

Plan: 1 to add, 0 to change, 0 to destroy.

Warning: Argument is deprecated

with aws_s3_bucket.s3_bucket,
on main.tf line 4, in resource "aws_s3_bucket" "s3_bucket":
4:     acl = "private"

acl is deprecated. Use the aws_s3_bucket_acl resource instead.

(and one more similar warning elsewhere)

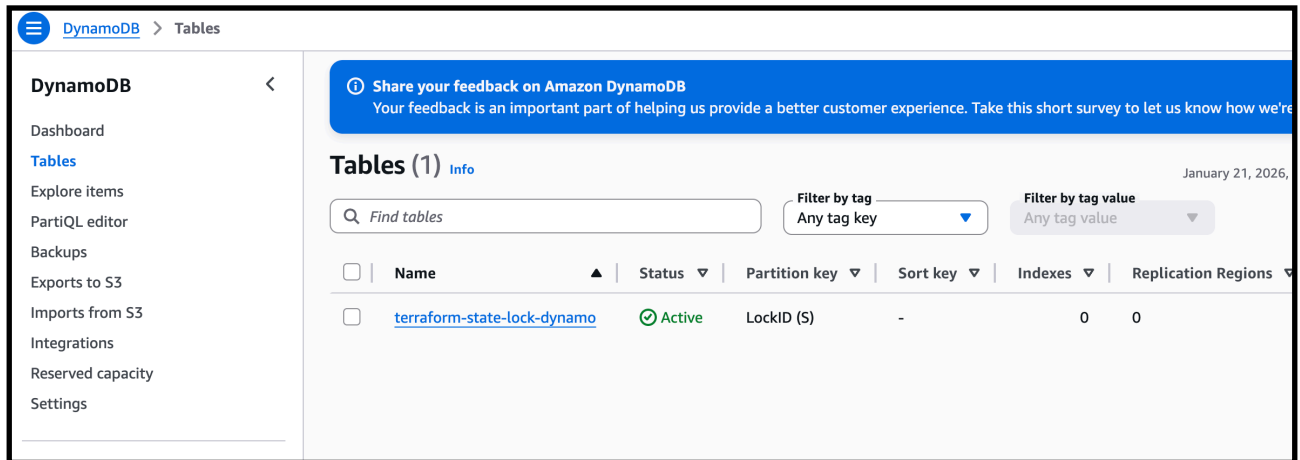
Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

aws_dynamodb_table.dynamodb-terraform-state-lock: Creating...
aws_dynamodb_table.dynamodb-terraform-state-lock: Still creating... [00m10s elapsed]
aws_dynamodb_table.dynamodb-terraform-state-lock: Creation complete after 15s [id=terraform-state-lock-dynamo]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
waseem:terraform-aws waseemakram$
```

—-> now check your aws account and look for dynamo db table



S3 as backend for terraform.tfstate file:

```
=====
terraform {
  backend "s3" {
    bucket = "s3backend"
    dynamodb_table = "terraform-state-lock-dynamo"
    key = "terraform.tfstate"
    region = "us-east-1"
  }
}
```

After adding above code

—> terraform init

—-> terraform apply

—-> and check the terraform state file . We can see the state file is empty because it is pushed in s3 bucket which we created

The image shows a VS Code editor with two tabs: `main.tf` and `terraform.tfstate`.

The `main.tf` file contains the following Terraform configuration:

```
resource "aws_dynamodb_table" "dynamodb-terraform-state-lock" {
  attribute {
    name = "LockID"
    type = "S"
  }
}

terraform {
  backend "s3" {
    bucket = "terraform-bucket-0880"
    dynamodb_table = "terraform-state-lock-dynamo"
    key = "terraform.tfstate"
    region = "us-east-1"
  }
}
```

The `terraform.tfstate` file contains the following content:

```
1 Generate code (*I), or select a language (*K M). Start typing to dismiss or don't show this again.
```

The **TERMINAL** tab shows the output of the `terraform init` command:

```
Successfully configured the backend "s3"! Terraform will automatically
use this backend unless the backend configuration changes.
Initializing provider plugins...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v6.28.0

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.

waseem:terraform-aws waseemakram$ terraform apply
Acquiring state lock. This may take a few moments...
aws_dynamodb_table.dynamodb-terraform-state-lock: Refreshing state... [id=terraform-state-lock-dynamo]
aws_s3_bucket.s3_bucket: Refreshing state... [id=terraform-bucket-0880]

No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and found no differences, so no changes are needed.

Warning: Argument is deprecated

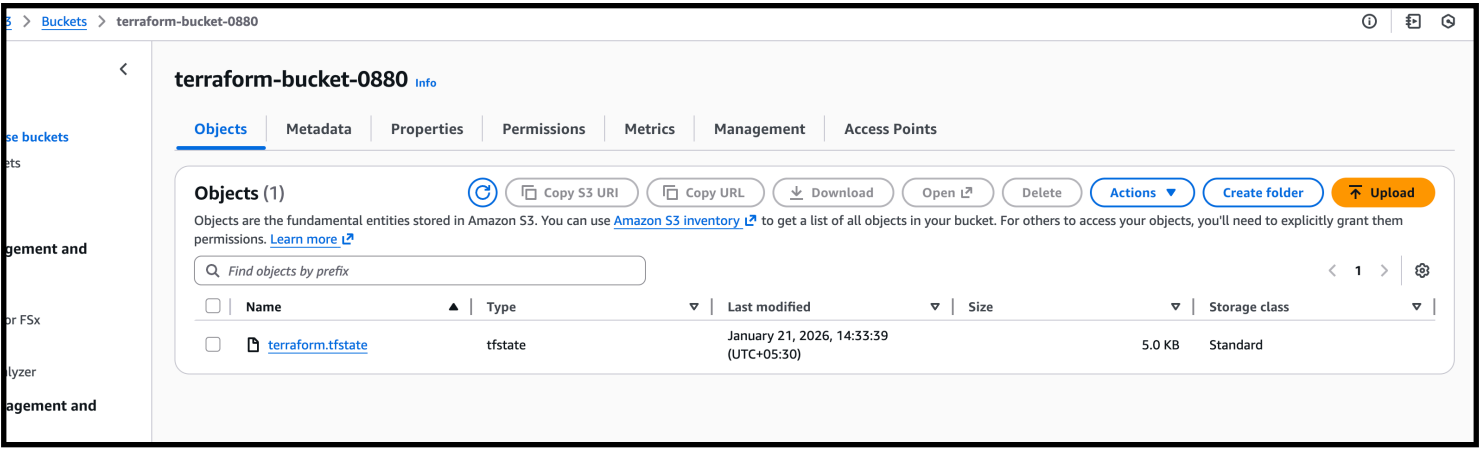
  with aws_s3_bucket.s3_bucket,
  on main.tf line 4, in resource "aws_s3_bucket" "s3_bucket":
   4:     acl = "private"

acl is deprecated. Use the aws_s3_bucket_acl resource instead.

(and one more similar warning elsewhere)

Releasing state lock. This may take a few moments...
```

—> check your bucket —> objects—> we will see the state files



Share your feedback on Amazon DynamoDB
Your feedback is an important part of helping us provide a better customer experience. Take this short survey to let us know how we're doing.

Edit item
You can add, remove, or edit the attributes of an item. You can nest attributes inside other attributes up to 32 levels deep. [Learn more](#)

Attributes

Attribute name	Value	Type	Remove
LockID - Partition key	terraform-bucket-0880/terraform.tfstate-md5	String	
Digest	93555a20d3958631493a618a92a67b88	String	Remove

Buttons: Add new attribute, Cancel, Save, Save and close

Terraform state commands:

=====

Terraform show --> To show the resources of the state file

Terraform state list --> List resources in the state

Terraform mv --> Move an item in the state

Terraform pull --> Pull current state and output to stdout

Terraform push --> Update remote state from a local state file

Terraform replace-provider --> Replace provider in the state

Terraform rm --> Remove instances from the state

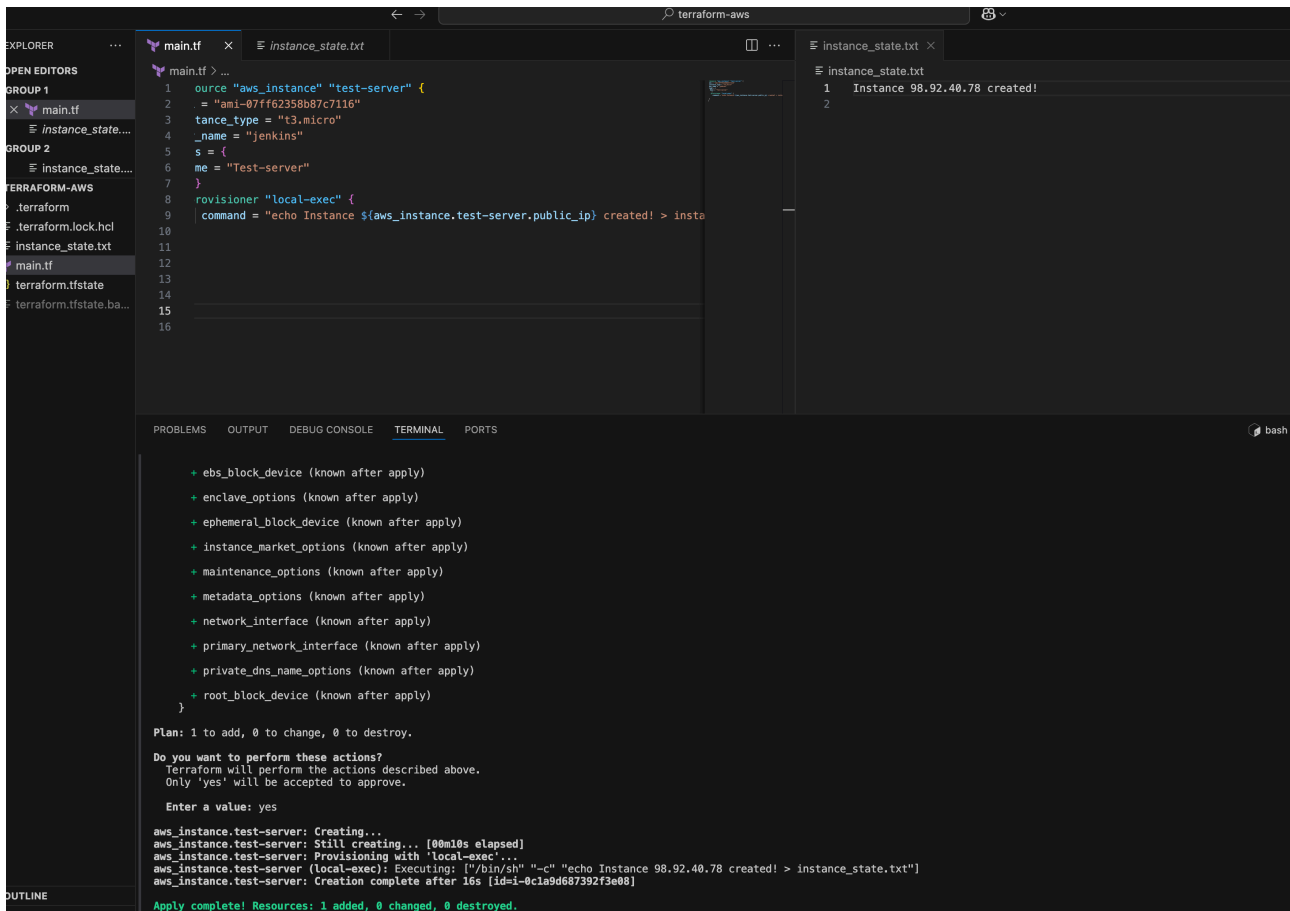
Terraform show --> Show a resource in the state

Terraform Provisioners:

=====

Terraform provisioners allow us to execute command, scripts on remote machines or local place where terraform is installed.

Provisioners will be written inside the resource blocks.



The screenshot shows a VS Code editor with a Terraform configuration file `main.tf` and a terminal window. The `main.tf` file contains the following code:

```
1 resource "aws_instance" "test-server" {
2   ami = "ami-07ff62358b87c7116"
3   instance_type = "t3.micro"
4   key_name = "jenkins"
5   tags = {
6     Name = "Test-server"
7   }
8   provisioner "local-exec" {
9     command = "echo Instance ${aws_instance.test-server.public_ip} created! > instance_state.txt"
10  }
11 }
12
13
14
15
16
```

The terminal window shows the output of the `terraform apply` command, including the plan, the execution of the `local-exec` provisioner, and the final state of the resources.

```
+ ebs_block_device (known after apply)
+ enclave_options (known after apply)
+ ephemeral_block_device (known after apply)
+ instance_market_options (known after apply)
+ maintenance_options (known after apply)
+ metadata_options (known after apply)
+ network_interface (known after apply)
+ primary_network_interface (known after apply)
+ private_dns_name_options (known after apply)
+ root_block_device (known after apply)
}

Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

aws_instance.test-server: Creating...
aws_instance.test-server: Still creating... [0m10s elapsed]
aws_instance.test-server: Provisioning with 'local-exec'...
aws_instance.test-server (local-exec): Executing: ["/bin/sh" "-c" "echo Instance 98.92.40.78 created! > instance_state.txt"]
aws_instance.test-server: Creation complete after 16s [id=i-8c1a9d687392f3e08]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
```

```
resource "aws_instance" "test-server" {
  ami = "ami-07ff62358b87c7116"
  instance_type = "t3.micro"
  key_name = "jenkins"
  tags = {
    Name = "Test-server"
  }
  provisioner "local-exec" {
    command = "echo Instance ${aws_instance.test-server.public_ip} created! > instance_state.txt"
  }
}
```

--> terraform init
--> terraform apply

--> now check the instance in aws

☐	Test-server	i-0c1a9d687392f3e08	Running	t3.micro	Initializing	View alarms +	us-east-1f
Select an instance							

1) Create at the time creating resource (Default)

The screenshot shows a code editor with a Terraform configuration file `main.tf` and a terminal window. The `main.tf` file contains a `resource "local_file" "test"` block with a `provisioner "local-exec"` block. The terminal shows the output of `terraform apply`, including the execution plan and the successful creation of the `local_file.test` resource.

```
main.tf > ...
1 resource "local_file" "test" {
2   filename = "tiger.txt"
3   content = "hello whitetiger"
4   provisioner "local-exec" {
5     command = "echo created the tiger.txt file > tiger.txt"
6   }
7 }
8
9
10
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
waseem:terraform-aws waseemakram$ terraform apply

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# local_file.test will be created
+ resource "local_file" "test" {
  + content          = "hello whitetiger"
  + content_base64sha256 = (known after apply)
  + content_base64sha512 = (known after apply)
  + content_md5       = (known after apply)
  + content_sha1      = (known after apply)
  + content_sha256    = (known after apply)
  + content_sha512    = (known after apply)
  + directory_permission = "0777"
  + file_permission   = "0777"
  + filename          = "tiger.txt"
  + id                = (known after apply)
}

Plan: 1 to add, 0 to change, 0 to destroy.

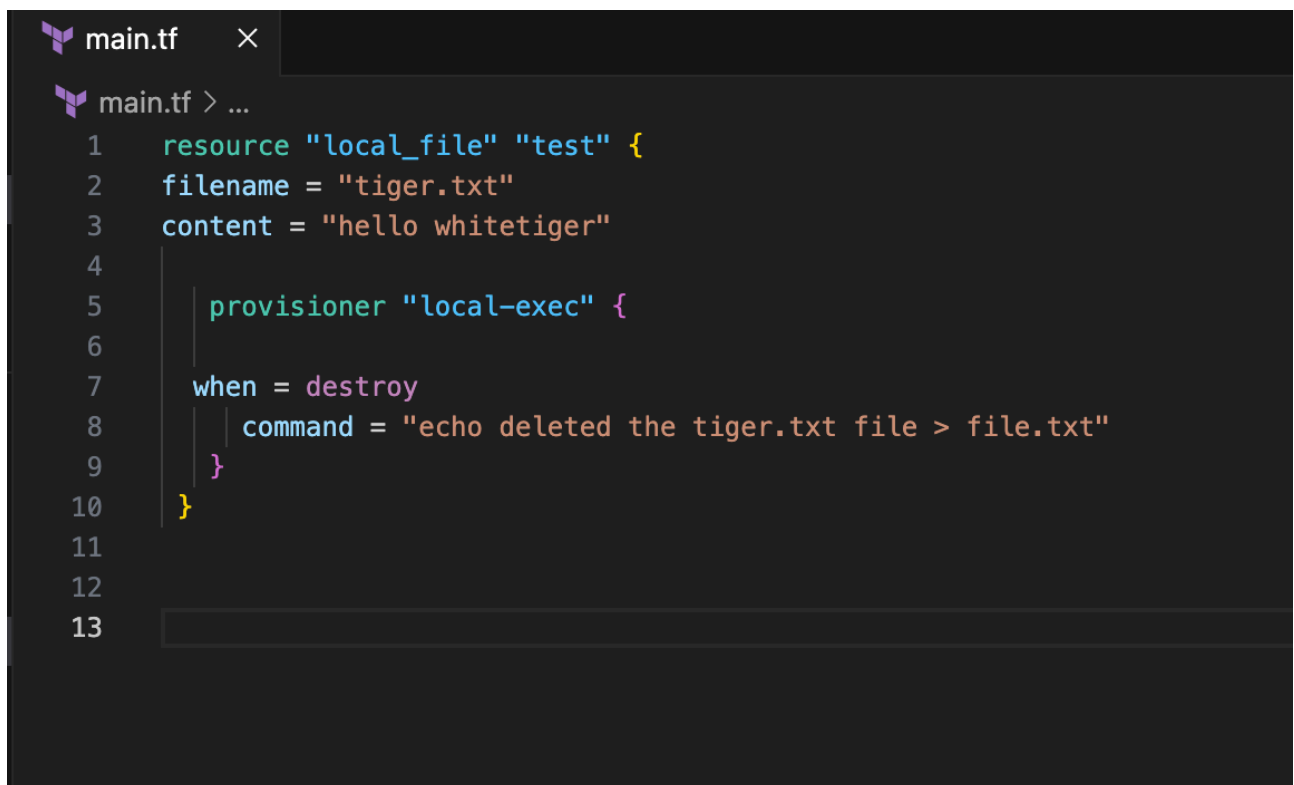
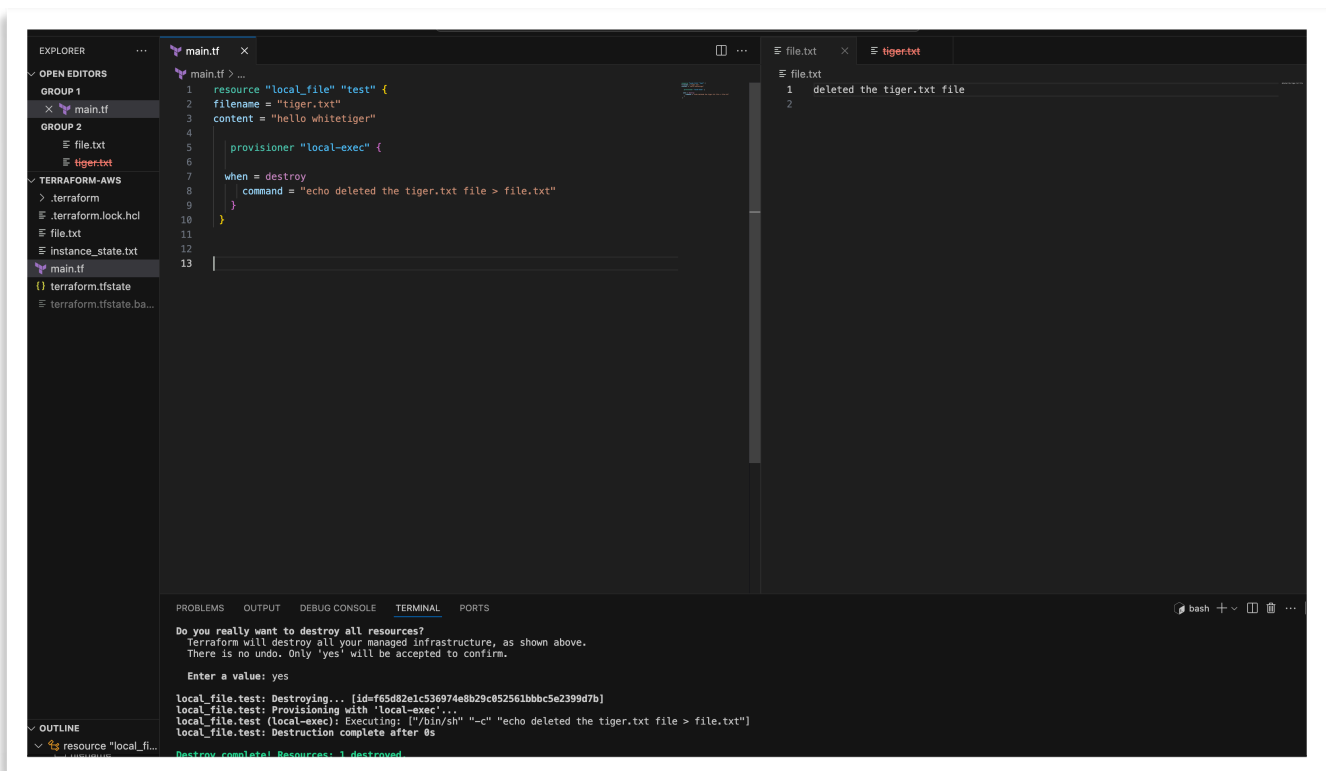
Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

local_file.test: Creating...
local_file.test: Provisioning with 'local-exec'...
local_file.test (local-exec): Executing: ["/bin/sh" "-c" "echo created the tiger.txt file > tiger.txt"]
local_file.test: Creation complete after 0s [id=f65d82e1c536974e8b29c052561bbbc5e2399d7b]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
waseem:terraform-aws waseemakram$
```

2) create at the time of destroyin resource (when = destroy)



—> terraform destroy

Terraform taint and untaint:

There would be cases when resource creation will get failed, if this happens then terraform will be marked as "Tainted".

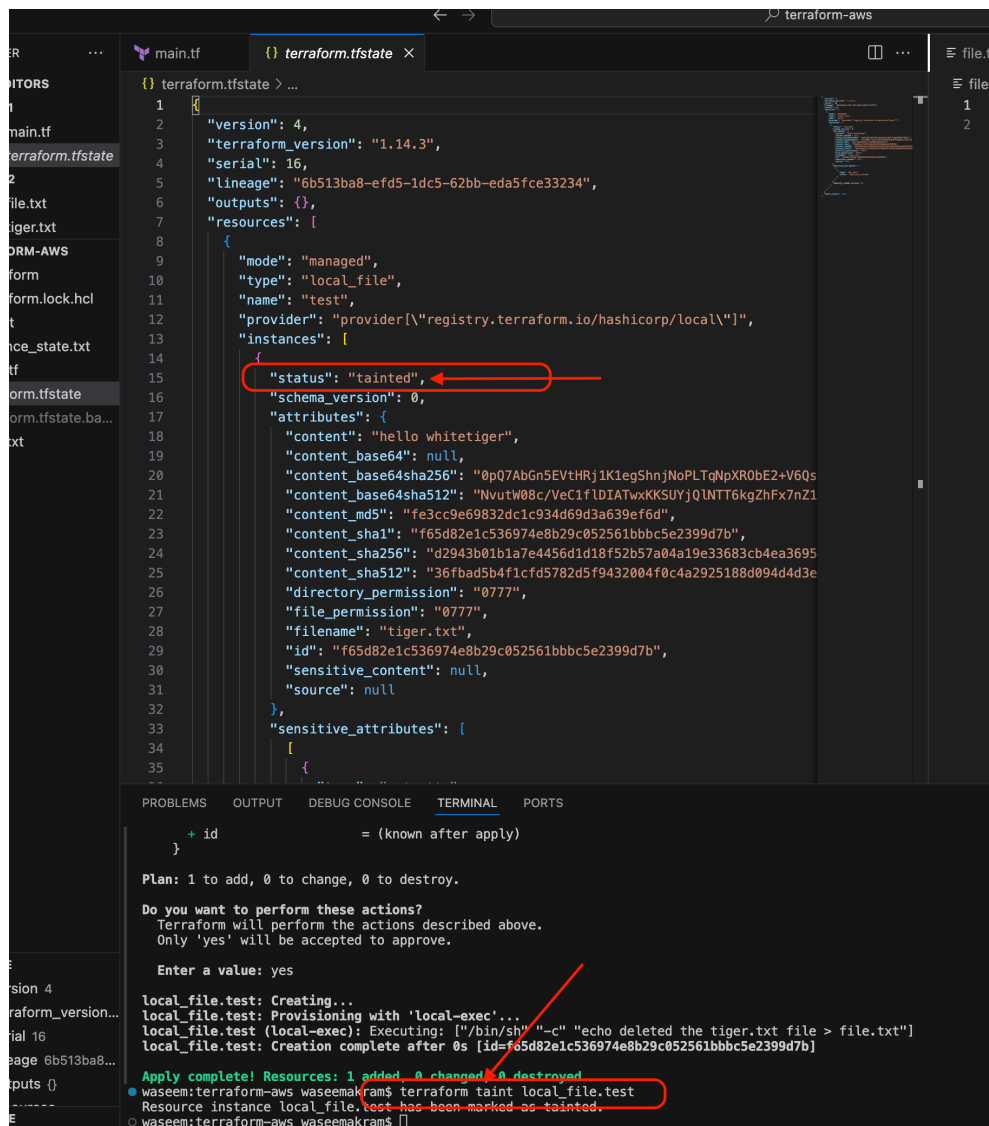
We can see this when we execute terraform plan command and this will be replaced when we use terraform apply command.

If for any reason you have installed manually in ec2 and you want that to be replaced in next apply

then we can manually mark that resource as taint in terraform so that it will be replaced next time when use apply.

terraform taint aws_instance.webserver

to undo the changes we can use untaint command.



```
1  {
2    "version": 4,
3    "terraform_version": "1.14.3",
4    "serial": 16,
5    "lineage": "6b513ba8-efd5-1dc5-62bb-eda5fce33234",
6    "outputs": {},
7    "resources": [
8      {
9        "mode": "managed",
10       "type": "local_file",
11       "name": "test",
12       "provider": "provider[\"registry.terraform.io/hashicorp/local\"]",
13       "instances": [
14         {
15           "status": "tainted",
16           "schema_version": 0,
17           "attributes": {
18             "content": "hello whitetiger",
19             "content_base64": null,
20             "content_base64sha256": "0pQ7AbGn5EVtHRj1K1egShnjNoPLTqNpXR0bE2+V6Qs",
21             "content_base64sha512": "NvutW08c/VeC1fLDIATwxKKSUYjQUNTT6kgZhFx7nZ1",
22             "content_md5": "fe3cc9e69832dc1c934d69d3a639ef6d",
23             "content_sha1": "f65d82e1c536974e8b29c052561bbbc5e2399d7b",
24             "content_sha256": "d2943b01b1a7e4456d1d18f52b57a04a19e33683cb4ea3695",
25             "content_sha512": "36fbad5b4f1cfd5782d5f9432004f0c4a2925188d094d4d3e",
26             "directory_permission": "0777",
27             "file_permission": "0777",
28             "filename": "tiger.txt",
29             "id": "f65d82e1c536974e8b29c052561bbbc5e2399d7b",
30             "sensitive_content": null,
31             "source": null
32           },
33           "sensitive_attributes": [
34             [
35               {
36                 "content": "hello whitetiger"
37               }
38             ]
39           ]
40         }
41       ]
42     }
43   ]
44 }
```

```
Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

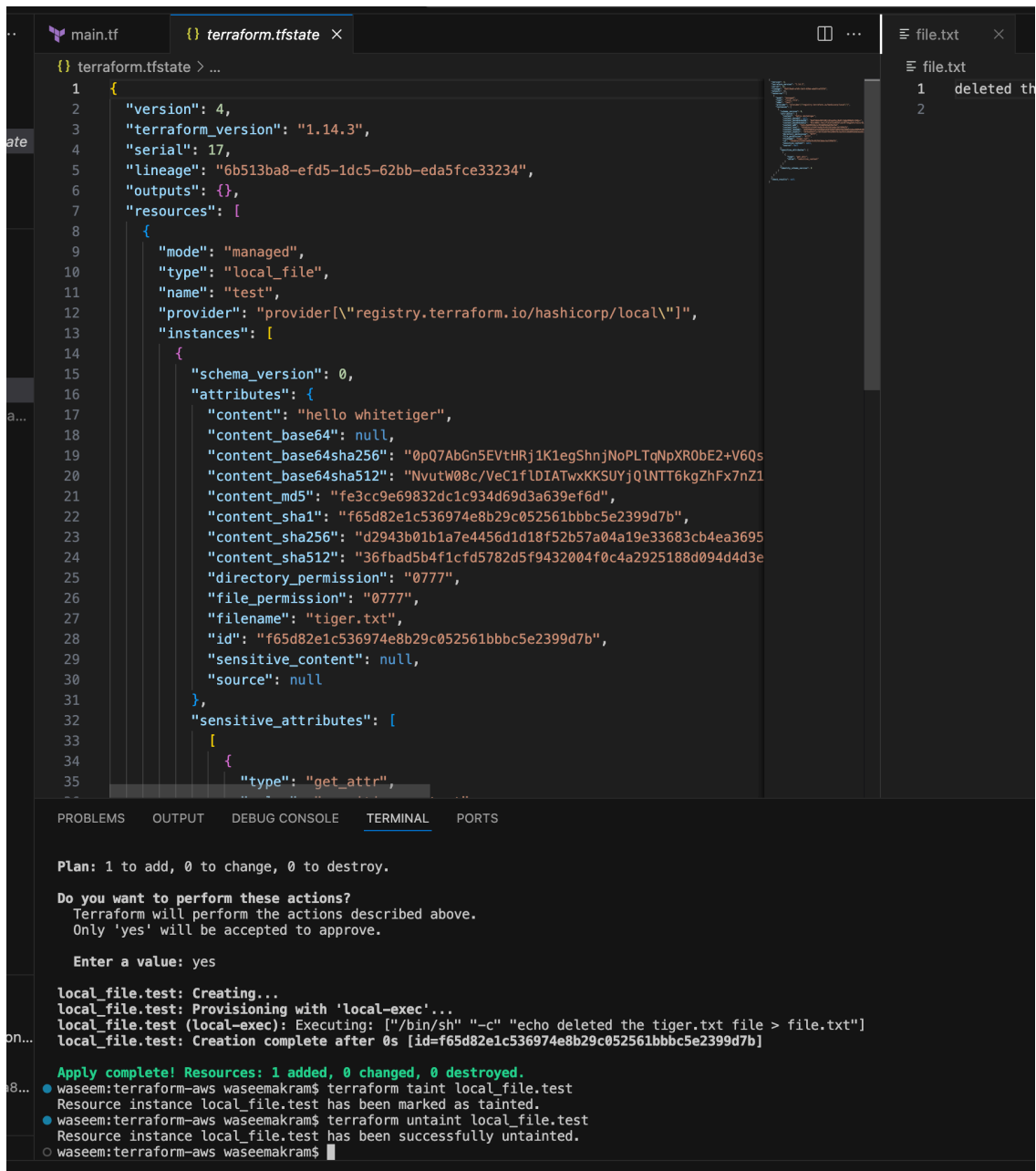
  Enter a value: yes

local_file.test: Creating...
local_file.test: Provisioning with 'local-exec'...
local_file.test (local-exec): Executing: ["/bin/sh" "-c" "echo deleted the tiger.txt file > file.txt"]
local_file.test: Creation complete after 0s [id=f65d82e1c536974e8b29c052561bbbc5e2399d7b]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed
● waseem:terraform-aws waseemakram$ terraform taint local_file.test
Resource instance local_file.test has been marked as tainted.
○ waseem:terraform-aws waseemakram$
```

terraform untaint aws_instance.webserver

this resource will not be created when using terraform apply.



The image shows a VS Code editor with a Terraform state file open. The state file is a JSON object representing the current state of the infrastructure. It includes metadata like version, serial, and lineage, and a list of resources. One resource, 'local_file.test', is shown in detail, including its mode, type, name, provider, and instances. The instances list contains a single entry for 'test' with various attributes like schema_version, attributes, and sensitive_attributes.

```
{
  "version": 4,
  "terraform_version": "1.14.3",
  "serial": 17,
  "lineage": "6b513ba8-efd5-1dc5-62bb-eda5fce33234",
  "outputs": {},
  "resources": [
    {
      "mode": "managed",
      "type": "local_file",
      "name": "test",
      "provider": "provider[\"registry.terraform.io/hashicorp/local\"]",
      "instances": [
        {
          "schema_version": 0,
          "attributes": {
            "content": "hello whitetiger",
            "content_base64": null,
            "content_base64sha256": "0pQ7AbGn5EVtHRj1K1egShnjNoPLTqNpXR0bE2+V6Qs",
            "content_base64sha512": "NvutW08c/VeC1fLDIATwxKKSUYjQLNTT6kgZhFx7nZ1",
            "content_md5": "fe3cc9e69832dc1c934d69d3a639ef6d",
            "content_sha1": "f65d82e1c536974e8b29c052561bbbc5e2399d7b",
            "content_sha256": "d2943b01b1a7e4456d1d18f52b57a04a19e33683cb4ea3695",
            "content_sha512": "36fbad5b4f1cfd5782d5f9432004f0c4a2925188d094d4d3e",
            "directory_permission": "0777",
            "file_permission": "0777",
            "filename": "tiger.txt",
            "id": "f65d82e1c536974e8b29c052561bbbc5e2399d7b",
            "sensitive_content": null,
            "source": null
          },
          "sensitive_attributes": [
            {
              "type": "get_attr",
            }
          ]
        }
      ]
    }
  ]
}
```

The terminal output shows the Terraform plan and apply process. It starts with a plan showing one resource to be added. Then, it asks for confirmation to perform the actions. After confirming, it shows the resource 'local_file.test' being created and provisioned. The output also shows the resource being marked as tainted and then successfully untainted.

```
Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

local_file.test: Creating...
local_file.test: Provisioning with 'local-exec'...
local_file.test (local-exec): Executing: ["/bin/sh" "-c" "echo deleted the tiger.txt file > file.txt"]
local_file.test: Creation complete after 0s [id=f65d82e1c536974e8b29c052561bbbc5e2399d7b]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
● waseem:terraform-aws waseemakram$ terraform taint local_file.test
Resource instance local_file.test has been marked as tainted.
● waseem:terraform-aws waseemakram$ terraform untaint local_file.test
Resource instance local_file.test has been successfully untainted.
○ waseem:terraform-aws waseemakram$
```

Debugging:

=====

terraform apply will provide us the logs/cause of the issue, but still if we want to dig deeper then we need to export a variable

```
export TF_LOG=TRACE
```

```
Set-Item -Path env:TF_LOG -value "TRACE"
```

Terraform provides 5 levels of logs:

- 1) INFO
- 2) WARNING
- 3) ERROR
- 4) DEBUG
- 5) TRACE

To store the logs permanently then we can export a path

Terraform import:

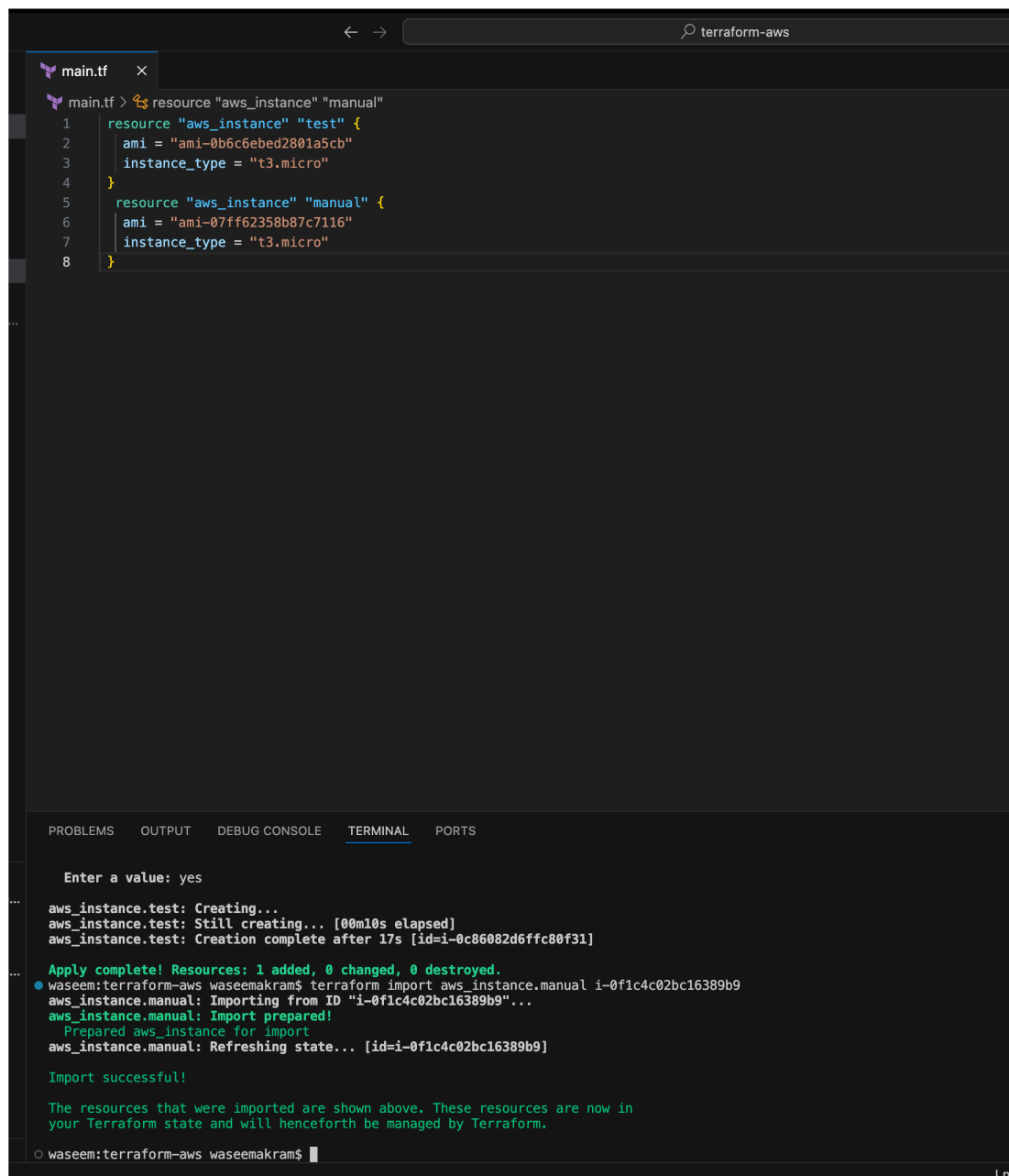
=====

Terraform import is used to import the existing infrastructure in terraform state file.

Once import is done then we can be able to create/delete and manage the infrastructure.

In order to import any resource we need to write the resource details in configuration file.

```
terraform import aws_instance.<name> instance_id
```



The image shows a VS Code editor window with a file named `main.tf`. The file contains two Terraform resource blocks for `aws_instance`. The first resource is named `test` and uses the AMI `ami-0b6c6ebd2801a5cb`. The second resource is named `manual` and uses the AMI `ami-07ff62358b87c7116`. Both resources are configured with `instance_type = "t3.micro"`.

Below the editor, the `TERMINAL` tab is active, showing the output of a Terraform command. The output indicates that the `aws_instance.test` resource was created successfully. Then, the `aws_instance.manual` resource was imported from the ID `i-0f1c4c02bc16389b9`. The terminal shows the following steps:

```
Enter a value: yes
aws_instance.test: Creating...
aws_instance.test: Still creating... [00m10s elapsed]
aws_instance.test: Creation complete after 17s [id=i-0c86082d6ffc80f31]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
waseem:terraform-aws waseemakram$ terraform import aws_instance.manual i-0f1c4c02bc16389b9
aws_instance.manual: Importing from ID "i-0f1c4c02bc16389b9"...
aws_instance.manual: Import prepared!
  Prepared aws_instance for import
aws_instance.manual: Refreshing state... [id=i-0f1c4c02bc16389b9]

Import successful!

The resources that were imported are shown above. These resources are now in
your Terraform state and will henceforth be managed by Terraform.

waseem:terraform-aws waseemakram$
```

-->terraform import aws_instance.manual
i-0f1c4c02bc16389b9

--> i-0f1c4c02bc16389b9 is the ami id of other instance
(manual)

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

Enter a value: yes

aws_instance.test: Creating...
aws_instance.test: Still creating... [00m10s elapsed]
aws_instance.test: Creation complete after 17s [id=i-0c86082d6ffc80f31]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
● waseem:terraform-aws waseemakram$ terraform import aws_instance.manual i-0f1c4c02bc16389b9
aws_instance.manual: Importing from ID "i-0f1c4c02bc16389b9"...
aws_instance.manual: Import prepared!
  Prepared aws_instance for import
aws_instance.manual: Refreshing state... [id=i-0f1c4c02bc16389b9]

Import successful!

The resources that were imported are shown above. These resources are now in
your Terraform state and will henceforth be managed by Terraform.

○ waseem:terraform-aws waseemakram$
```

☐	i-0c86082d6ffc80f31	Running	t3.micro	3/3 checks passed	View alarms	us-east-1f	ec2-13-221-197-143.co...	13.221.197.1
☒	testing	Running	t3.micro	Initializing	View alarms	us-east-1f	ec2-98-93-188-152.co...	98.93.188.15

i-0f1c4c02bc16389b9 (testing)

Now —> terraform destroy

```
waseem:terraform-aws waseemakram$ terraform destroy
aws_instance.manual: Refreshing state... [id=i-0f1c4c02bc16389b9]
aws_instance.test: Refreshing state... [id=i-0c86082d6ffc80f31]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
- destroy

Terraform will perform the following actions:
```

```
# aws_instance.manual will be destroyed
- resource "aws_instance" "manual" {
  - ami                  = "ami-07ff62358b87c7116" -> null
  - arn                  = "arn:aws:ec2:us-east-1:207662791773:instance/i-0f1c4c02bc16389b9" -> null
  - associate_public_ip_address = true -> null
  - availability_zone      = "us-east-1f" -> null
  - disable_api_stop       = false -> null
  - disable_api_termination = false -> null
  - ebs_optimized          = true -> null
  - force_destroy          = false -> null
  - get_password_data       = false -> null
  - hibernation             = false -> null
  - id                     = "i-0f1c4c02bc16389b9" -> null
  - instance_initiated_shutdown_behavior = "stop" -> null
  - instance_state          = "running" -> null
  - instance_type           = "t3.micro" -> null
  - ipv6_address_count       = 0 -> null
  - ipv6_addresses           = [] -> null
  - key_name                 = "jenkins" -> null
  - monitoring               = false -> null
  - placement_partition_number = 0 -> null
  - primary_network_interface_id = "eni-0eadc722e35f4594e" -> null
  - private_dns              = "ip-172-31-68-116.ec2.internal" -> null
  - private_ip               = "172.31.68.116" -> null
  - public_dns               = "ec2-98-93-188-152.compute-1.amazonaws.com" -> null
  - public_ip                = "98.93.188.152" -> null
  - region                   = "us-east-1" -> null
  - secondary_private_ips     = [] -> null
  - security_groups           = [
    - "launch-wizard-9",
  ] -> null
  - source_dest_check         = true -> null
  - subnet_id                 = "subnet-04b85ff06e8cc53be" -> null
  - tags                      = {
    - "Name" = "testing"
  } -> null
  - tags_all                  = {
    - "Name" = "testing"
  } -> null
  - tenancy                   = "default" -> null
  - vpc_security_group_ids     = [
    - "sg-0a31a20e1a88efc5f",
  ] -> null
}
```

Ln 8, Col 3 Spaces: 2 UTF

☐	i-0c86082d6ffc80f31	⌚ Shutting-d...	🔍 ⓘ t3.micro	-	View alarms +	us-east-1f	ec2-13-221-197-143.co...	13.221.197.1
☒	testing	i-0f1c4c02bc16389b9	⌚ Shutting-d...	🔍 ⓘ t3.micro	-	View alarms +	us-east-1f	ec2-98-93-188-152.co... 98.93.188.15

Task 3:

3. Create one EC2 instance with httpd installed using a Terraform script.

—> step 1: first add aws configure in your terraform folder

—> now In your terraform folder add main.tf file

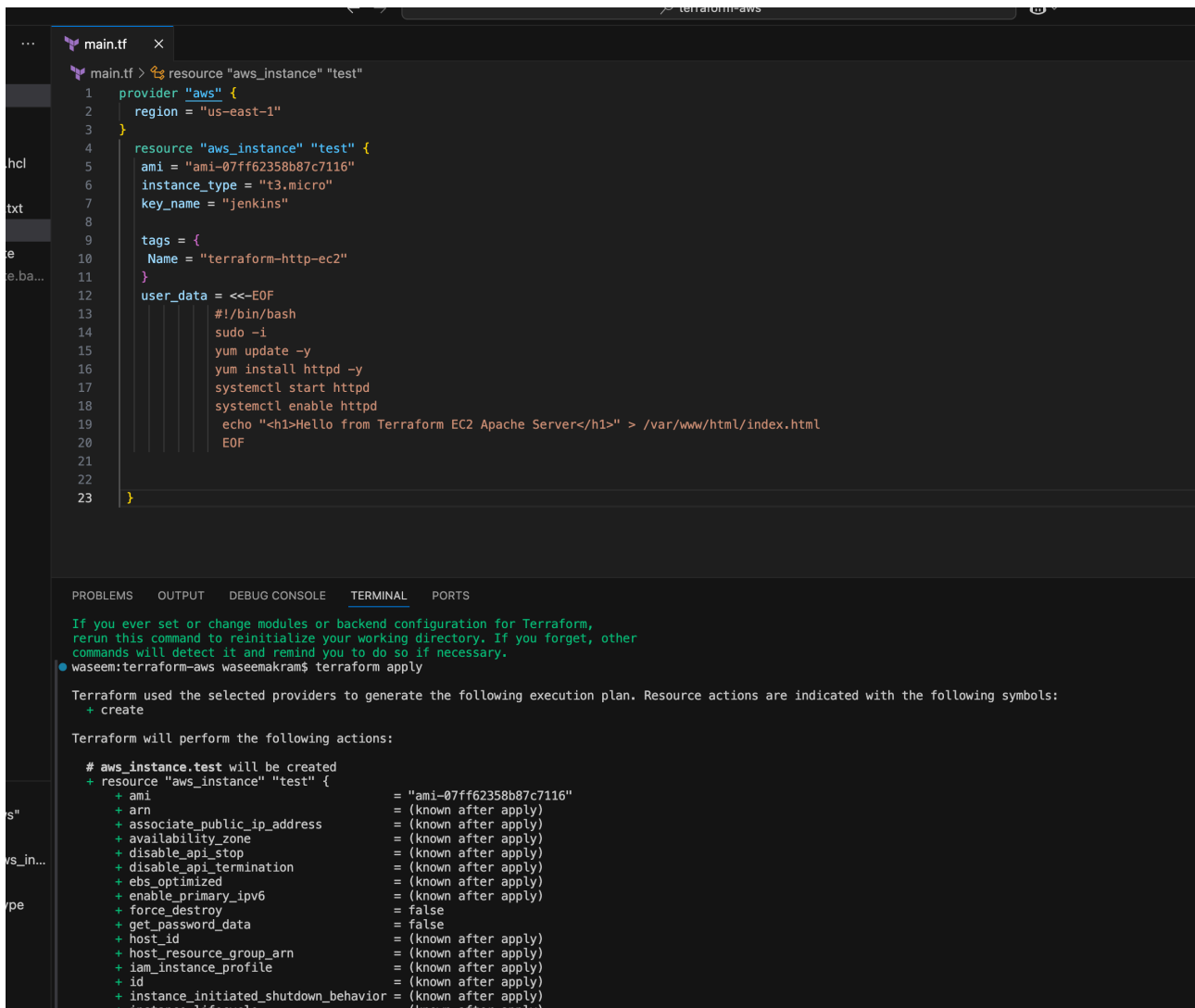
—> and add below code

```
main.tf x
main.tf > resource "aws_instance" "test"
1  provider "aws" {
2    region = "us-east-1"
3  }
4  resource "aws_instance" "test" {
5    ami = "ami-07ff62358b87c7116"
6    instance_type = "t3.micro"
7    key_name = "jenkins"
8
9    tags = {
10     Name = "terraform-http-ec2"
11   }
12   user_data = <<--EOF
13   |         #!/bin/bash
14   |         sudo -i
15   |         yum update -y
16   |         yum install httpd -y
17   |         systemctl start httpd
18   |         systemctl enable httpd
19   |         echo "<h1>Hello from Terraform EC2 Apache Server</h1>" > /var/www/html/index.html
20   |         EOF
21
22
23 }
```

—> provide the ami = amazon machine image details

—> key_name = your pem key name

—> after adding code



```
main.tf
1 provider "aws" {
2   region = "us-east-1"
3 }
4 resource "aws_instance" "test" {
5   ami = "ami-07ff62358b87c7116"
6   instance_type = "t3.micro"
7   key_name = "jenkins"
8
9   tags = {
10     Name = "terraform-http-ec2"
11   }
12   user_data = <<-EOF
13     #!/bin/bash
14     sudo -i
15     yum update -y
16     yum install httpd -y
17     systemctl start httpd
18     systemctl enable httpd
19     echo "<h1>Hello from Terraform EC2 Apache Server</h1>" > /var/www/html/index.html
20   EOF
21 }
22
23
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

If you ever set or change modules or backend configuration for Terraform, rerun this command to reinitialize your working directory. If you forget, other commands will detect it and remind you to do so if necessary.

waseem:terraform-aws waseemakram\$ terraform apply

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:

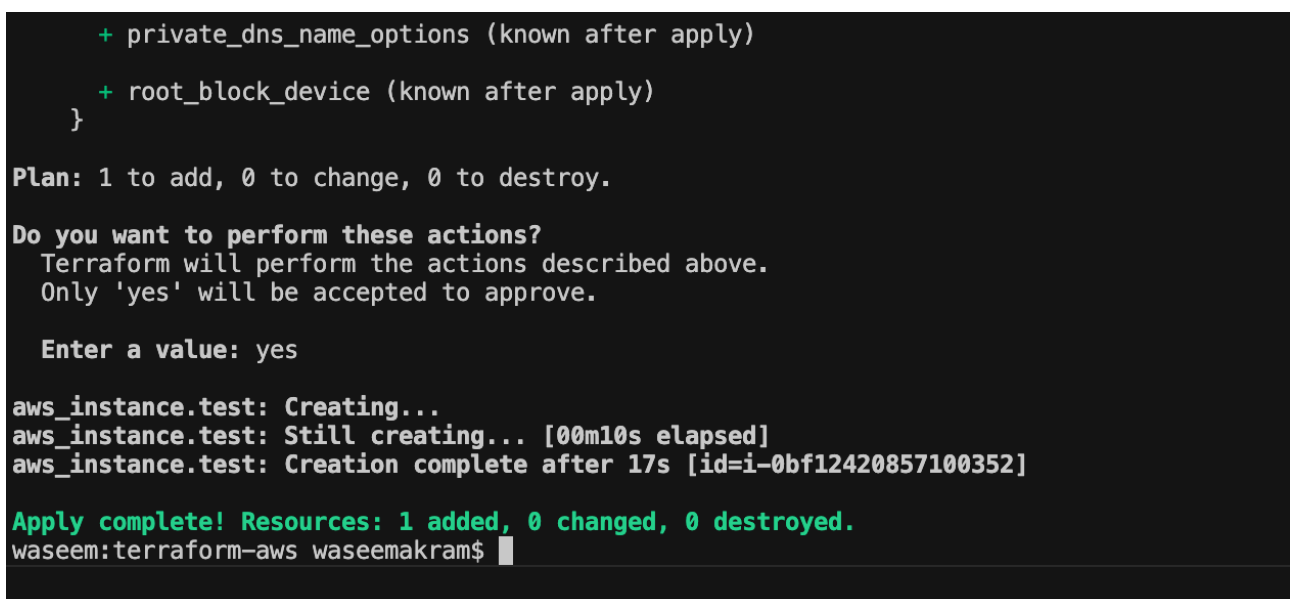
- + create

Terraform will perform the following actions:

```
# aws_instance.test will be created
+ resource "aws_instance" "test" {
+   ami              = "ami-07ff62358b87c7116"
+   arn              = (known after apply)
+   associate_public_ip_address = (known after apply)
+   availability_zone = (known after apply)
+   disable_api_stop  = (known after apply)
+   disable_api_termination = (known after apply)
+   ebs_optimized     = (known after apply)
+   enable_primary_ipv6 = (known after apply)
+   force_destroy     = false
+   get_password_data = false
+   host_id           = (known after apply)
+   host_resource_group_arn = (known after apply)
+   iam_instance_profile = (known after apply)
+   id               = (known after apply)
+   instance_initiated_shutdown_behavior = (known after apply)
+   instance_lifecycle = (known after apply)
```

—-> terraform init —> run this command

—-> terraform apply



```
+ private_dns_name_options (known after apply)
+ root_block_device (known after apply)
}

Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

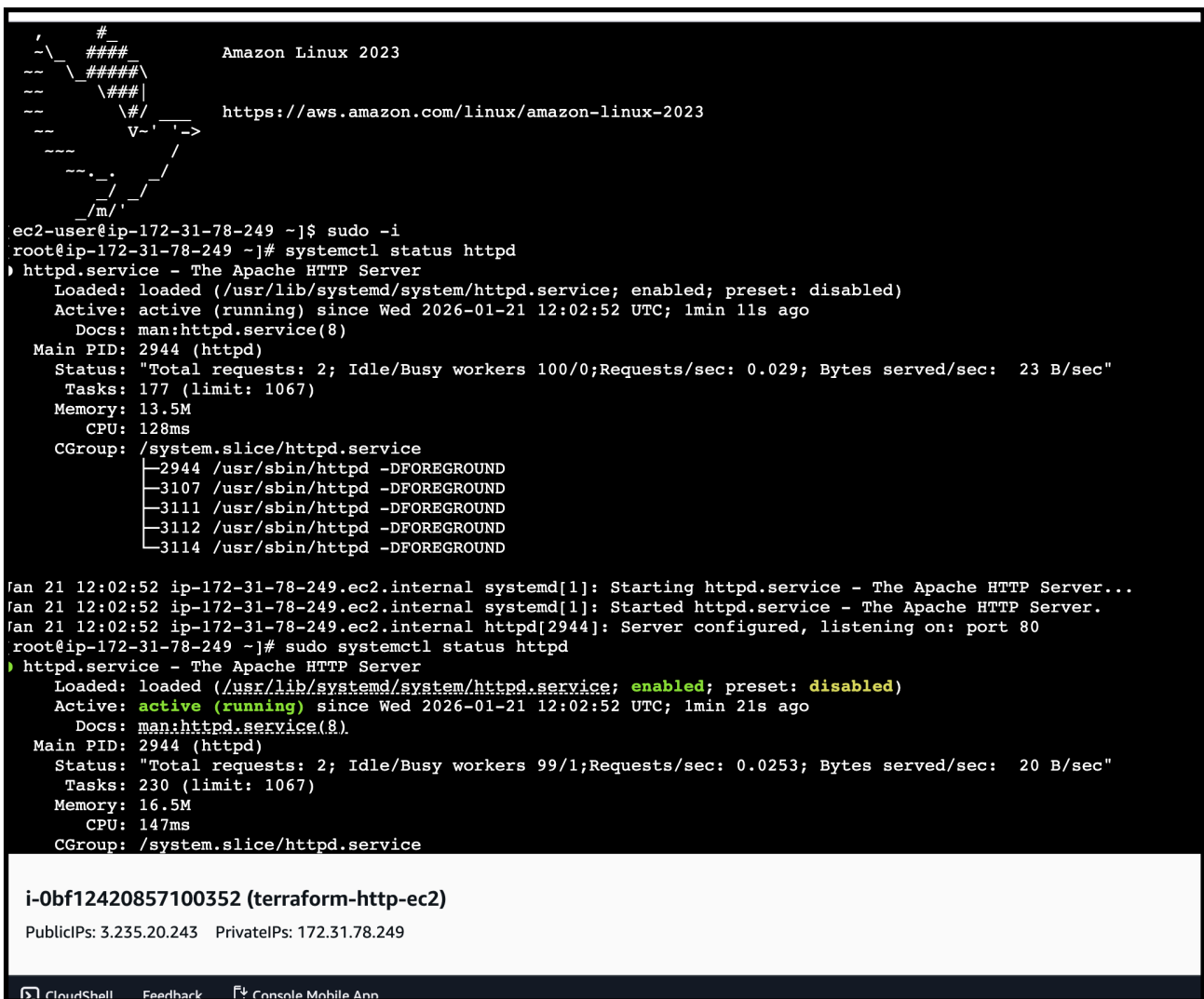
Enter a value: yes

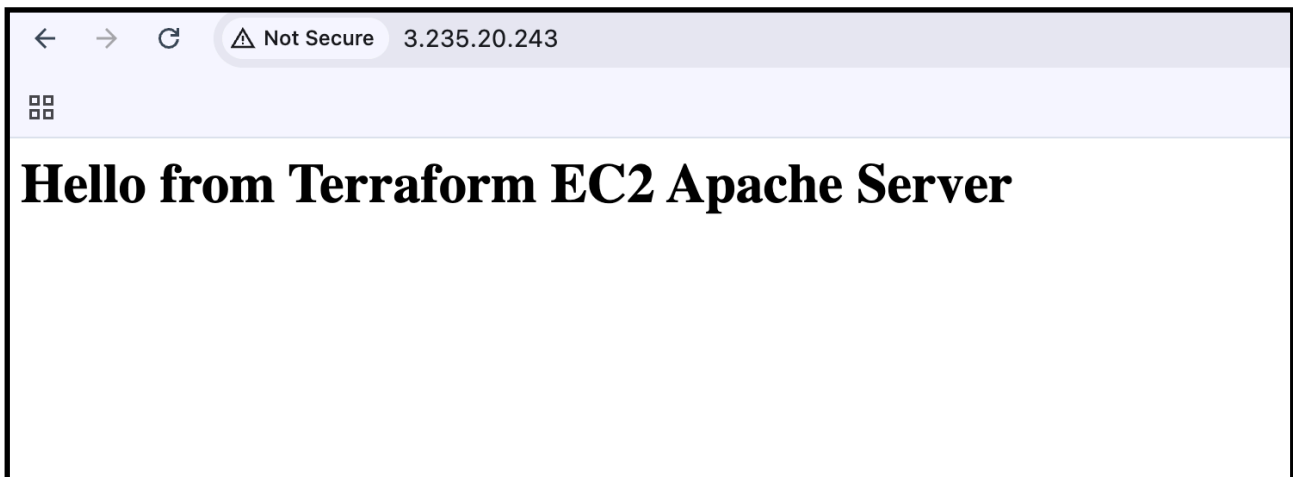
aws_instance.test: Creating...
aws_instance.test: Still creating... [00m10s elapsed]
aws_instance.test: Creation complete after 17s [id=i-0bf12420857100352]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
waseem:terraform-aws waseemakram$
```

--> check your aws instances

--> check the httpd is running or not





```
systemctl enable httpd  
echo "<h1>Hello from Terraform EC2 Apache Server</h1>" > /var/www/html/index.html  
EOF
```

—> we can see same data is shown in server

Task 4:

4. Set up **S3 as backend** for task

—>add your available bucket name

```
terraform {  
  backend "s3" {  
    bucket = "waseem-s3-terraform-bucket02"  
    key    = "task3/terraform.tfstate"  
    region = "us-east-1"  
  }  
}
```

The image shows a VS Code editor window with a Terraform configuration file named `main.tf` and its terminal output.

main.tf

```
1 provider "aws" {
2   region = "us-east-1"
3 }
4
5 terraform {
6   backend "s3" {
7     bucket = "waseem-s3-terraform-bucket02"
8     key    = "task3/terraform.tfstate"
9     region = "us-east-1"
10  }
11 }
12
13 resource "aws_instance" "test" {
14   ami           = "ami-07ff62358b87c7116"
15   instance_type = "t3.micro"
16   key_name      = "jenkins"
17
18   tags = {
19     Name = "terraform-http-ec2"
20   }
21
22   user_data = <<-EOF
23   | #!/bin/bash
24   | sudo -i
25   | yum update -y
26   | yum install httpd -y
27   | systemctl start httpd
28   | systemctl enable httpd
29   | echo "<h1>Hello from Terraform EC2 Apache Server</h1>" > /var/www/html/index.html
30   | EOF
31 }
32
33 }
```

Terminal Output

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
● waseem:terraform-aws waseemakram$ terraform apply
aws_instance.test: Refreshing state... [id=i-0bf12420857100352]

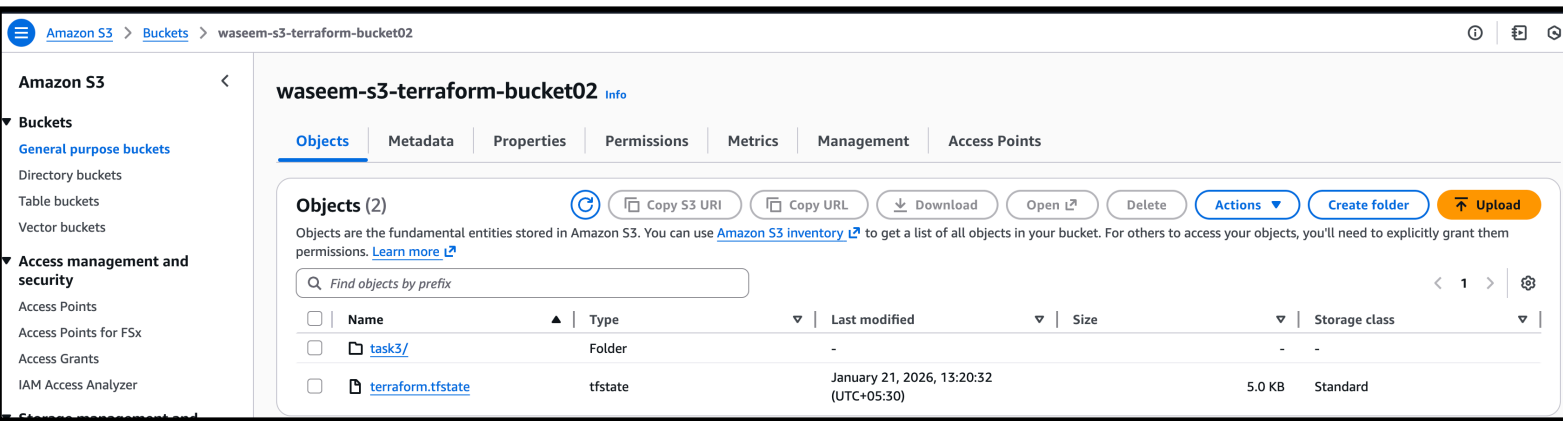
No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and found no differences, so no changes are needed.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.
○ waseem:terraform-aws waseemakram$
```

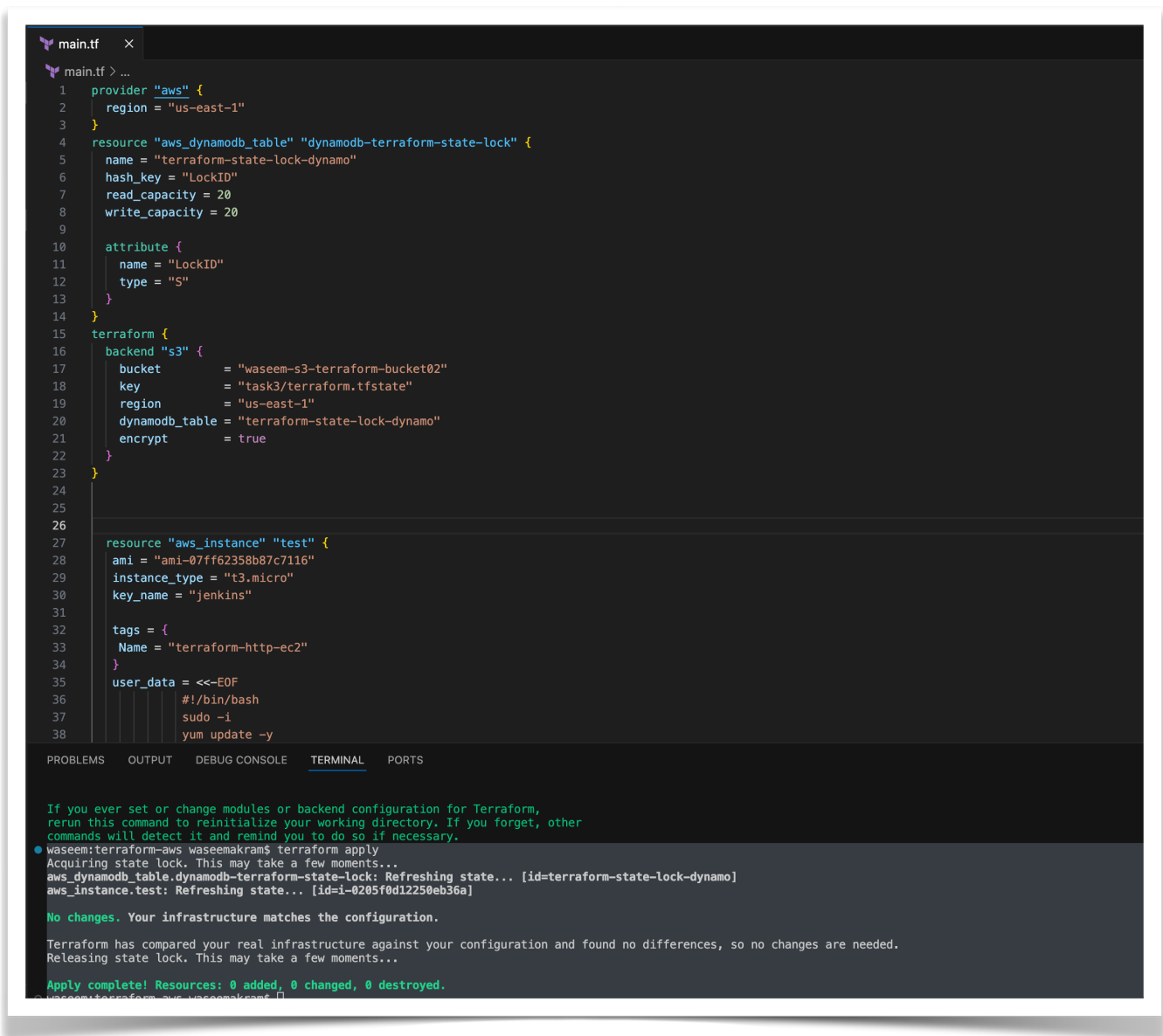
—> now apply

—-> check your s3 bucket



Task 5:

5.Set up DynamoDB locking for task 3.



```
resource "aws_dynamodb_table" "dynamodb-terraform-state-lock" {
  name = "terraform-state-lock-dynamo"
  hash_key = "LockID"
  read_capacity = 20
  write_capacity = 20

  attribute {
    name = "LockID"
    type = "S"
  }
}

terraform {
  backend "s3" {
    bucket      = "waseem-s3-terraform-bucket02"
    key         = "task3/terraform.tfstate"
    region      = "us-east-1"
    dynamodb_table = "terraform-state-lock-dynamo"
    encrypt     = true
  }
}
```

--> add the above details code

--> create one db and later add that data base in side the s3 backend

--> and apply

--> now check your dynamic db

DynamoDB > Explore items: terraform-state-lock-dynamo > Edit item

Share your feedback on Amazon DynamoDB

Your feedback is an important part of helping us provide a better customer experience. Take this short survey to let us know how we're doing.

Share feedback

Edit item

You can add, remove, or edit the attributes of an item. You can nest attributes inside other attributes up to 32 levels deep. [Learn more](#)

FormJSON view

Attributes

Add new attribute

Attribute name	Value	Type	Remove
LockID - Partition key	waseem-s3-terraform-bucket02/task3/terraform.tfstate-md5	String	
Digest	4baba444f16633125e0ac6d718178eb5	String	Remove

Cancel

Save

Save and close

-->

Warm throughput

Info

Last updated January 21, 2026, 18:56 (UTC+5:30)

Edit

Prepare your table for planned peak events, without impacting your application performance or availability. Learn more about [Amazon DynamoDB pricing](#)

Name	Status	Type	Read units per second	Write operations per second
terraform-state-lock-dynamo	Active	Table	20	20

Deletion protection

Info

Turn on

—> s3 bucket objects

Amazon S3 > Buckets > waseem-s3-terraform-bucket02

waseem-s3-terraform-bucket02

Info

Objects

Metadata

Properties

Permissions

Metrics

Management

Access Points

Objects (2)

Copy S3 URI

Copy URL

Download

Open

Delete

Actions

Create folder

Upload

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

Find objects by prefix

< 1 >

	Name	Type	Last modified	Size	Storage class
	task3/	Folder	-	-	-
	terraform.tfstate	tfstate	January 21, 2026, 13:20:32 (UTC+05:30)	5.0 KB	Standard

Default Behavior

- By default, Terraform stores the state file (**terraform.tfstate**) **locally** inside your project folder.
- This state file is Terraform's **memory** of what resources exist in AWS.
- Problem: If you change laptop / multiple people run Terraform, state can mismatch and create **duplicate resources**.

Task 3: Create EC2 instance with httpd installed

We created **1 EC2 instance** using Terraform and installed **Apache httpd** using `user_data`.

What we did:

- Created EC2 instance (`t3.micro`)
- Used Amazon Linux AMI
- Added key pair for SSH login (`key_name = "jenkins"`)
- Installed and started Apache automatically

User Data Script (Auto Install httpd)

- `yum update -y`
- `yum install -y httpd`
- `systemctl start httpd`
- `systemctl enable httpd`
- Created a webpage:
Hello from Terraform EC2 Apache Server

Output: When we opened EC2 public IP in browser, the page worked.

Task 4: Setup S3 as Backend (Remote State)

We configured Terraform to store the **terraform.tfstate** file in **S3** instead of local system.

S3 Backend Details

- **Bucket:** `waseem-s3-terraform-bucket02`
- **State file path (key):** `task3/terraform.tfstate`
- **Region:** `us-east-1`

What this means:

- State is saved in S3
- Even if we change system, Terraform can still access the state
- Team members can use the same state safely

Task 5: Setup DynamoDB for Locking (State Lock)

We created a **DynamoDB table** for state locking to prevent 2 users from running Terraform at the same time.

DynamoDB Details

- **Table name:** `terraform-state-lock-dynamo`
- **Partition key:** `LockID` (String)

What this does:

- Before Terraform apply, it creates a lock entry in DynamoDB
- So nobody else can apply changes at same time
- After apply completes, lock is removed automatically

Proof: We saw in terminal:

- **Acquiring state lock**
- **Releasing state lock**

And also we checked DynamoDB items showing LockID and Digest.

Final Result (What we achieved)

- ✓ EC2 instance created with Apache running
- ✓ State stored in S3 remote backend
- ✓ DynamoDB locking enabled for safe apply